

Enhancement Two: Algorithms and Data Structures

Artifact Description

The artifact I selected for this enhancement is the Travlr Getaways full-stack web application, which I originally created in CS-465 Full Stack Development. The application allows users to browse travel packages, create accounts, make reservations, and manage travel information through both customer-facing and administrative interfaces. The system uses a Node.js and Express backend, MongoDB for data storage, and Angular for the administrative portal.

I selected this project because it is one of the most comprehensive applications I developed during the Computer Science program. The project contains multiple interconnected layers, including a frontend interface, backend API, controllers, services, and database operations. Because the application already had a strong foundation, it provided an opportunity to enhance the way data is processed and presented without requiring major architectural changes.

Enhancement Description

For this enhancement, I focused on improving the way trip data is organized, processed, and presented to users. The original application retrieved all available trip records from MongoDB and displayed them within the Angular administrative interface. While this approach functioned correctly for a small dataset, it would become increasingly inefficient and difficult to navigate as the number of travel packages increased.

To improve usability and scalability, I implemented search, filtering, and sorting functionality. Users can now search for trips using keywords, filter results by resort, maximum price, and trip duration, and sort trips by attributes such as name, price, duration, and start date. These enhancements allow users to quickly locate relevant travel packages instead of manually reviewing every available record.

The enhancement demonstrates practical applications of algorithms and data structures through the manipulation of collections of Trip objects stored in arrays. Search functionality performs comparisons between user-provided keywords and trip properties to identify matching records. Filtering operations iterate through the Trip collection and evaluate multiple conditions to determine which records satisfy the selected criteria. Sorting operations reorganize the resulting collection based on user-selected attributes, allowing information to be displayed in a more meaningful order.

A key design decision involved centralizing data processing within the backend rather than placing all processing logic inside Angular components. User-selected criteria are collected in

the frontend and transmitted through service classes to the Express API. Controllers process the Trip collection, apply the requested search, filtering, and sorting operations, and return only the appropriate results. This approach reduces frontend complexity, promotes code reuse, and creates a clearer separation between presentation logic and business logic.

Although the dataset currently contains a relatively small number of travel packages, these enhancements establish a foundation that can better support future growth. Organizing data processing logic into dedicated backend components improves maintainability while making it easier to extend functionality as application requirements evolve.

Alignment to Course Outcomes

This enhancement supports Course Outcome 3 because it focuses on improving an existing computing solution through the application of algorithms and data structures. The enhancement required analyzing how Trip objects were stored, retrieved, and processed before implementing a more efficient and user-friendly solution. Search, filtering, and sorting functionality improved the usefulness of the data while demonstrating algorithmic problem-solving skills.

This enhancement also supports Course Outcome 4 because it required the design, implementation, testing, and integration of new functionality within an existing full-stack application. The solution involved coordinating changes across Angular components, service classes, API routes, and backend controllers while maintaining compatibility with the existing system architecture. These changes improved both usability and maintainability without disrupting existing functionality.

Progress Toward Planned Outcomes

My original goal was to improve how trip information was processed and presented to users while demonstrating the practical application of algorithms and data structures. Through the implementation of search, filtering, and sorting functionality, I successfully achieved these objectives.

The completed enhancement closely aligns with the enhancement plan developed during the code review process. The application now provides users with efficient ways to locate and organize travel packages while maintaining a clean separation between frontend and backend responsibilities. The addition of these features significantly improves the overall user experience while preserving the application's original design.

At this time, I do not anticipate making significant changes to my outcome-coverage plan because the completed enhancement effectively demonstrates the application of algorithms and data structures within a real-world software project.

Reflection

This enhancement reinforced the idea that relatively simple algorithms can have a significant impact on application usability. While the original system successfully displayed travel packages, users had limited options for locating specific information. Adding search, filtering, and sorting functionality transformed the application into a more practical and user-friendly system.

One of the most valuable lessons I learned was how data flows through a full-stack application. User input originates within Angular components, passes through service classes, travels through API routes, and is ultimately processed within backend controllers before results are returned to the user. Working through this process strengthened my understanding of how different architectural layers interact to support application functionality.

Another important lesson involved scalability. Although filtering and sorting operations currently process relatively small collections of Trip objects, I recognized that larger datasets would require more efficient approaches. This experience helped me understand the importance of selecting appropriate algorithms and designing systems that can continue to perform effectively as data volumes increase.

Testing played an important role throughout the enhancement process. After implementing the new functionality, I tested numerous combinations of search terms, resort filters, price limits, trip durations, and sorting options to verify that accurate results were returned. Additional testing was performed to ensure multiple filters could operate simultaneously without producing incorrect results. Several debugging sessions were required to verify that frontend inputs, service calls, API routes, and controller logic were all processing data correctly.

Overall, this enhancement strengthened my understanding of algorithms, data structures, software architecture, and testing practices. The completed functionality improves the user experience while providing a scalable and maintainable solution for processing travel package data. It also demonstrates my ability to apply algorithmic thinking to practical software development challenges within a full-stack application.